

THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

# Retrieval Augmented Generation and Memory

# Part 1 · Retrieval

# Motivation: Much useful information is not in an LLM's pretraining corpus



Materials property databases



MRI images of brain injuries



Catalogs of astrophysical events



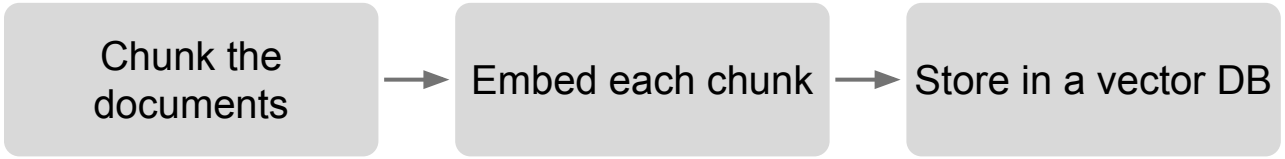
Your lab's notebooks, code, and internal docs

# The question: how can we make this data accessible to an agent?

- Fetch relevant documents at question time and add to the prompt
- Ground answers in your own data instead of only the model's training
- Keep knowledge current without retraining the model
- Reduce hallucinations and cite our sources

# Retrieval Augmented Generation (RAG) lifecycle

**OFFLINE: indexing (done once)**




**ONLINE: query**






THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

# The distributional hypothesis

*“You shall know a word by the company it keeps.”*

J. R. Firth, 1957

### Core idea

Words that occur in the same contexts tend to have similar meanings (Harris, 1954).

### Method

Count how often tokens co-occur, e.g. within sentences, and treat the counts as a vector.

### Impact

One of the most successful ideas in NLP. Modern embedding models learn the same signal at scale.

# Co-occurrence encodes relationships

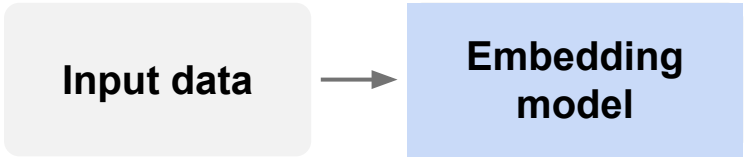
Scanning a large corpus, we might find:

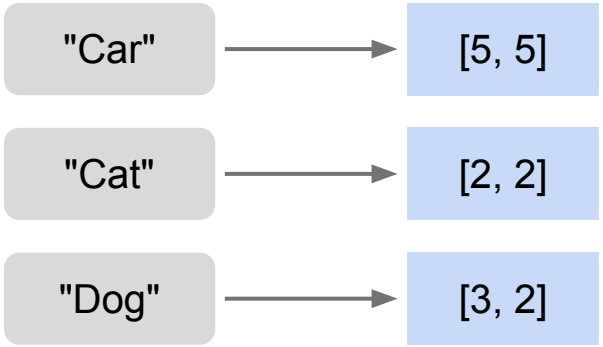
| | | | |
|---|---|---|---|
| **Bill Gates** | Microsoft | philanthropy | **billionaire** |
| **Warren Buffett** | investor | fortune | **billionaire** |

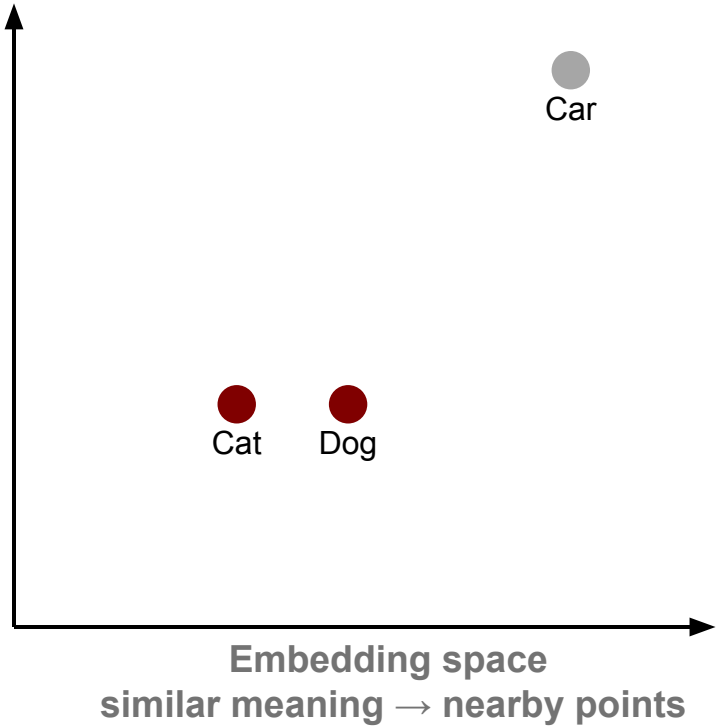
The shared context ("billionaire") pulls the two names close together in vector space

**Caution**: the same mechanism learns societal bias.

# What we mean by embeddings



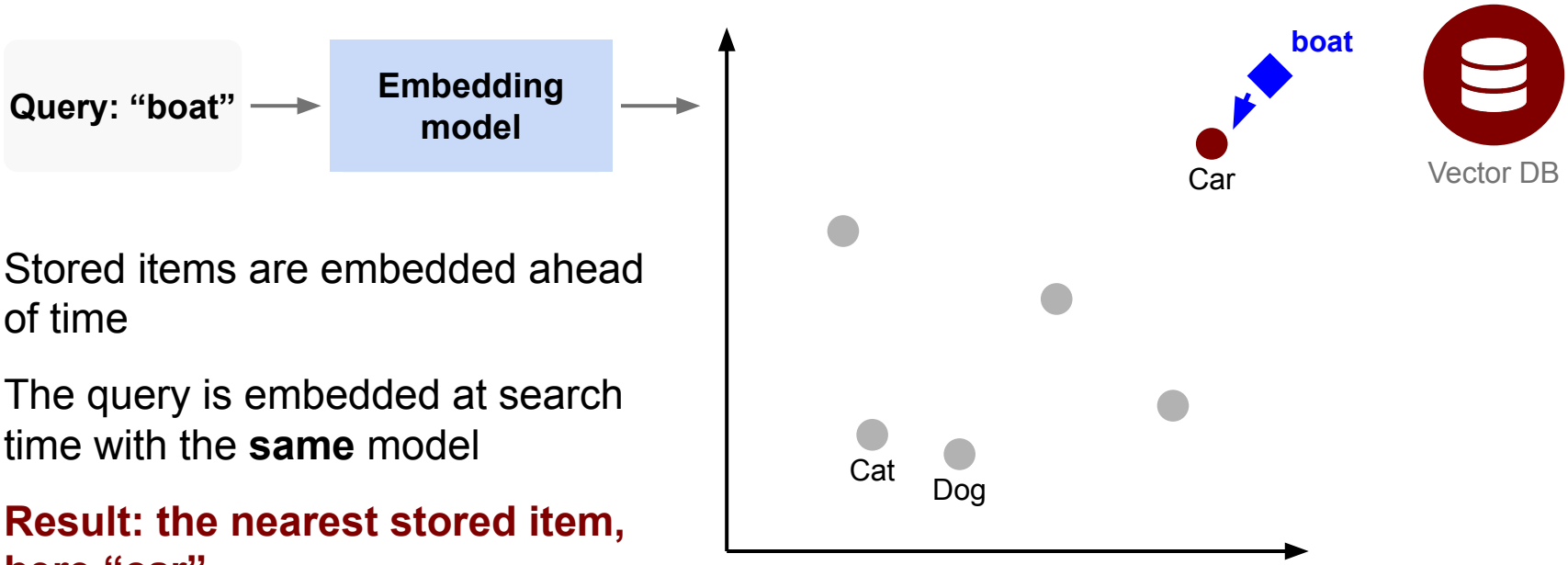






THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

# Retrieval of nearest neighbors




Stored items are embedded ahead of time

The query is embedded at search time with the **same** model

**Result: the nearest stored item, here “car”**



THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

# Measuring similarity

## Cosine

$$\cos\theta = a \cdot b / (\|a\| \, \|b\|)$$

Compares direction only. Ignores vector length. Higher is more similar.

## Euclidean

$$\|a - b\|$$

Straight-line distance between the points. Lower is more similar.

## Dot product

$$a \cdot b = \Sigma \, a_i b_i$$

Direction and length together. Cheapest to compute.

In practice: most text embedding models are trained for cosine. Normalize vectors to unit length and all three give the same ranking, so systems often normalize and then use the cheap dot product.

# Choosing the right metric

Using our toy vectors what is most similar to **Cat [2,2]**?

| Metric | Dog [3, 2] | Car [5, 5] | Closest |
|---|---|---|---|
| Cosine (higher = closer) | 0.98 | 1.00 | **Car** |
| Euclidean (lower = closer) | 1.00 | 4.24 | **Dog** |
| Dot product (higher = closer) | 10 | 20 | **Car** |

**Cat and Car point in the same direction (in our small toy embedding space)**

so cosine considers this an exact match

# Store and query: libraries vs. vector databases

### Index library: FAISS

Fast, in-memory index building and search. You handle persistence, updates, and filtering yourself. Great for experiments and batch jobs.

https://ai.meta.com/tools/faiss/

### Vector databases

Purpose-built: Qdrant, Milvus, Weaviate, Pinecone, Chroma. Added to existing stores: pgvector (Postgres), Elasticsearch.

https://qdrant.tech/
https://milvus.io/
https://weaviate.io/

# Scaling search?

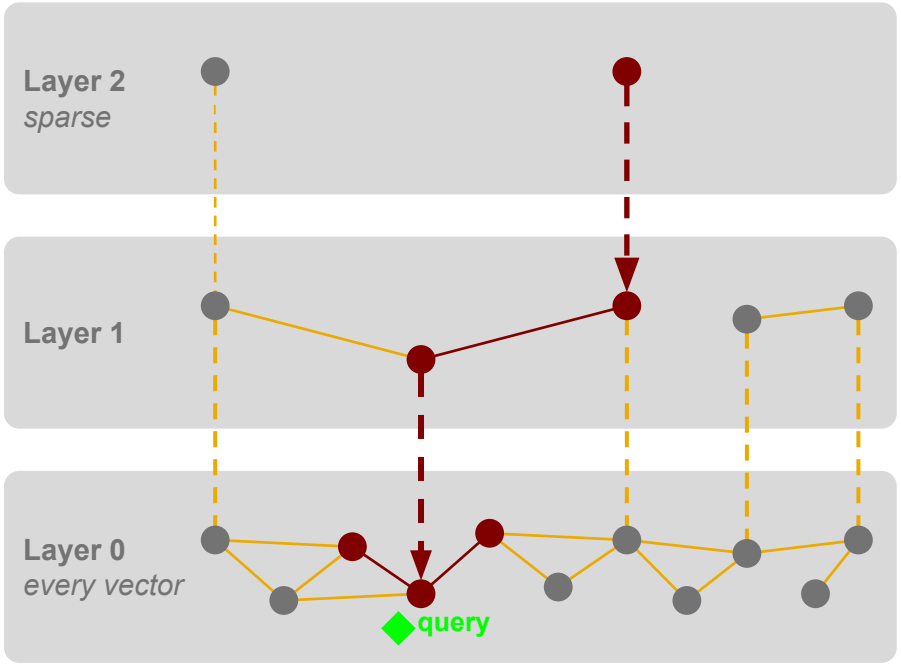
We have to do pairwise comparisons between a query and all stored embeddings

- O(N x d) per query

**Approximate nearest neighbor (ANN) search gives up a little recall for orders-of-magnitude speed improvement**

| | |
|---|---|
| **Trees** | Partition the space (k-d trees) |
| **Hashing** | Similar vectors collide in the same bucket (LSH) |
| **Clusters + quantization** | Search only nearby clusters; compress vectors (inverted file index, product quantization) |
| **Graphs** | Walk a graph of neighbors (HNSW) → next slide |

# Hierarchical Navigable Small World (HNSW)




*Malkov & Yashunin, IEEE TPAMI, 2020*

## How a query runs

1. Enter at the top, sparse layer
2. Greedily hop to the neighbor closest to the query
3. Drop down a layer and repeat
4. Search locally at layer 0 → top-k

**Knobs:** M (links per node), efConstruction (build quality), efSearch (recall vs. speed)

**Tradeoff:** fast, high-recall queries; costs memory and build time



THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

# Embedding functions and modalities

Each embedding model defines its own space. An image vector from one model can't be compared with a text vector from another.

### Joint space

Train text and images together (CLIP-style contrastive learning), so “a photo of a dog” lands near dog photos.

### One modality

Convert everything to text: caption the images, describe the tables, then embed the text.

### Separate + fuse

Keep one index per modality, search each, and merge the ranked lists.

# Research example: Multi-objective retrieval for heterogeneous scientific modalities



*"What percentage of red galaxies have stellar masses above $10^{11}$ solar masses?"*

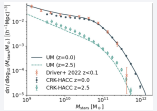
**Jordan**
cosmology researcher



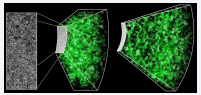

**Recent papers**
literature review



**Galaxy images**
image archive



**GSWLC catalog data**
galaxy catalog



**HACC simulations**
simulation outputs

## Why not one global index?

**Scale**
Vector collections reach sizes where centralizing them is impractical.

**Governance**
Data sits behind separate institutional boundaries.

**Placement**
Archives are split by instrument, project, or modality.

Credit: Song Oh

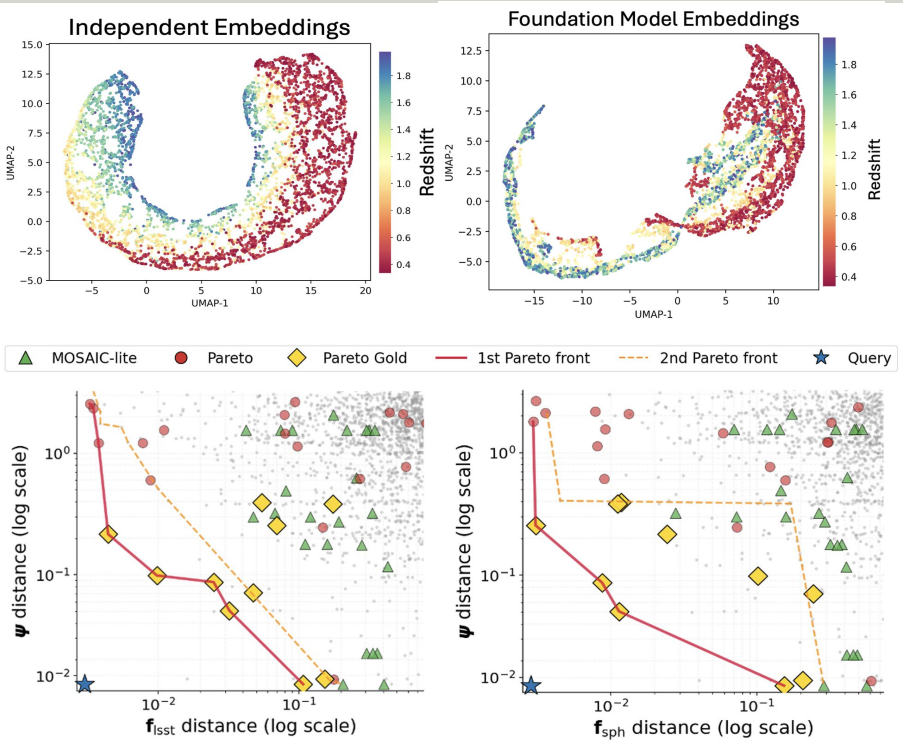
# Research example: Multi-objective retrieval for heterogeneous scientific modalities

Can post-hoc fusion of embeddings serve as an alternative to multimodal FMs for retrieval?

- **ParetoFuse**: embed each modality separately, then rank candidates with Pareto optimality instead of a single distance
- Keeps per-modality distances, so scientists can see which physical signals drive similarity

**Setup and Results**

- Corpus: 5,000 galaxies; 1,000 galaxy queries
- Independent embeddings: train one autoencoder per modality on 20K samples
- FM embeddings: lightweight version of the MOSAIC model
- Result: ParetoFuse's nearest neighbors are 8% closer to the query in raw-data space

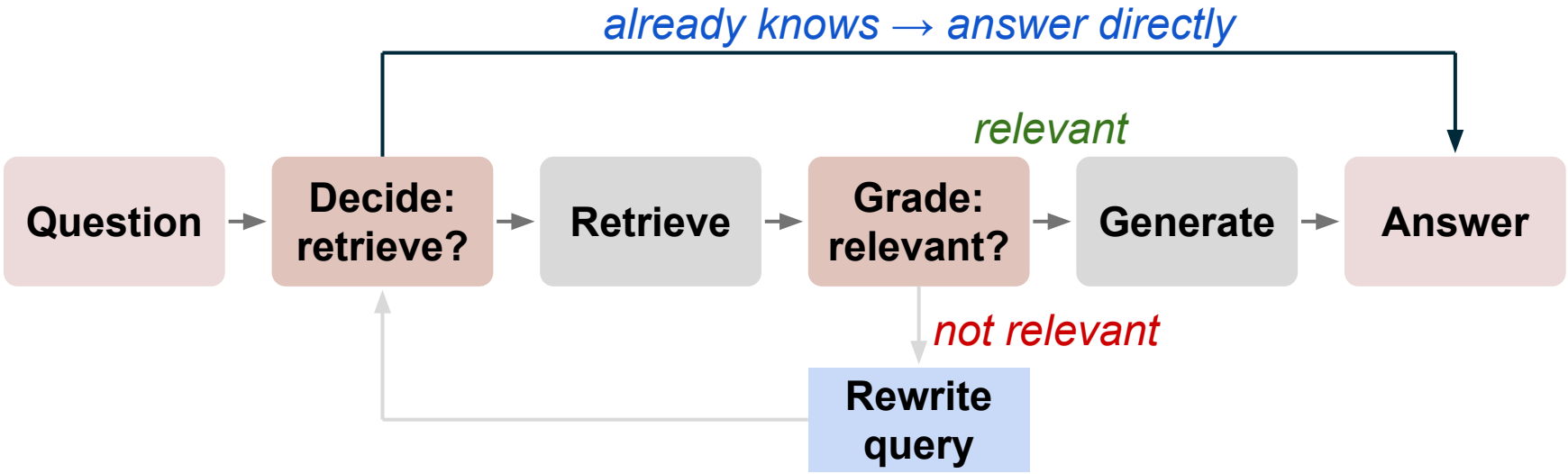



Credit: Song Oh



THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

# Agentic RAG



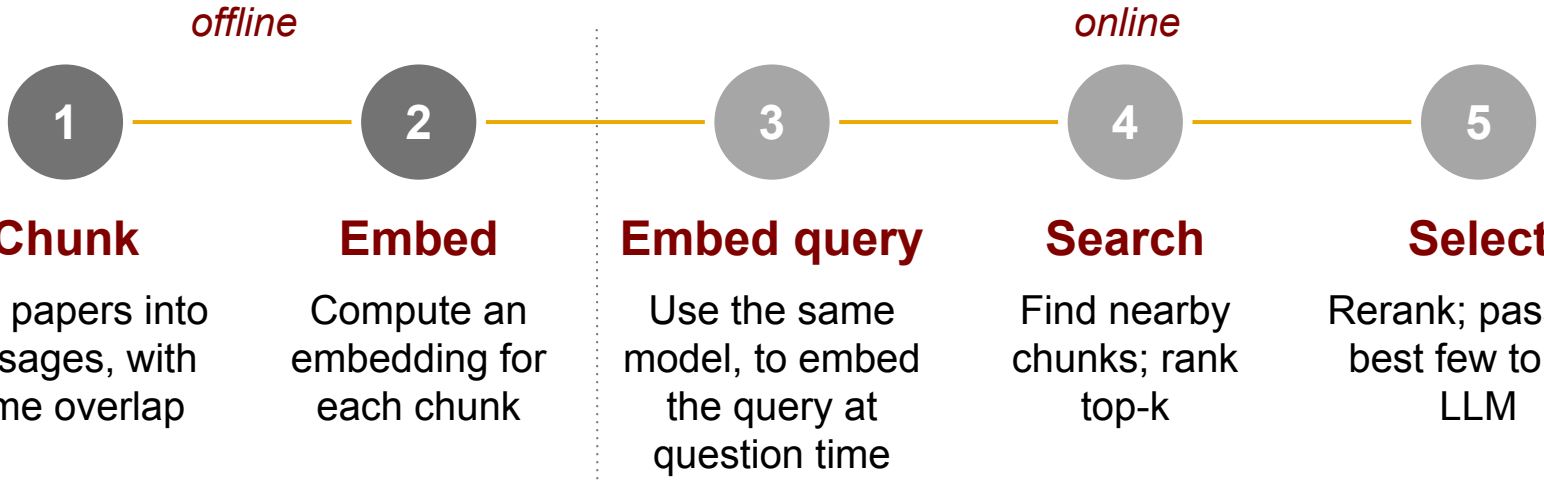

**Why:** naive RAG retrieves on every query, even “what's 2 + 2?”, and irrelevant chunks can make answers worse. Here retrieval is a tool the agent chooses to call.

*Walkthrough: docs.langchain.com/oss/python/langgraph/agentic-rag*



THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

# Activity 4: Indexing papers




**Tuning**
Chunk size and overlap · which embedding model · value of k



THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

# Detour: Measuring retrieval quality

- Answers are capped by retrieval quality, so we should measure retrieval individually
- *recall@k*: did the correct source appear in the top *k* results?
- Example: correct paper in the top 5 for 16 of 20 questions → *recall@5* = 0.8
- Needs a labeled set: each question paired with the paper or passage that answers it (e.g., our Q&A benchmark)
- Use it to tune: change chunk size, embedding model, or k, and see whether recall goes up

# Activity 4: RAG over papers

**Part 1: Vector database tool**

- Chunk and embed papers, then store them in Chroma
- Write a search tool and run it on our paper Q&A benchmark

**Part 2: RAG through MCP**

- Connect the Chroma MCP server to an agent and ingest the papers
- Ask your agent questions about the papers

# Chroma DB: A lightweight vector database

```
import sys
import chromadb

query = ' '.join(sys.argv[1:])

client = chromadb.Client()
collection = client.create_collection('science')

collection.add(
    ids=['1', '2', '3', '4'],
    documents=['Photosynthesis converts sunlight into chemical energy in plants.',
        'Black holes are regions where gravity is so strong that light cannot escape.',
        'DNA carries the genetic instructions for living organisms.',
        'Supernovae are powerful explosions at the end of a star\'s life.', ],)

results = collection.query(query_texts=[query], n_results=2)
print(results['documents'][0])
```

# Ingesting

```
def ingest(papers: dict[str, str], embed_client: OpenAI, embed_model: str) ->
chromadb.Collection:
    collection = chromadb.EphemeralClient().create_collection(
        'papers', embedding_function=None, configuration={'hnsw': {'space': 'cosine'}},
    )
    for name, text in papers.items():
        chunks = chunk_text(text)
        collection.add(
            ids=[f'{name}:{i}' for i in range(len(chunks))],
            documents=chunks,
            embeddings=embed(embed_client, embed_model, chunks, DOCUMENT_PREFIX),
            metadatas=[{'paper': name}] * len(chunks),
        )
    return collection
```

# Searching

```
def search_papers(ctx: RunContextWrapper[SearchContext], query: str, k: int = 5) ->
str:
    sc = ctx.context
    query_vector = embed(sc.embed_client, sc.embed_model, [query], QUERY_PREFIX)
    results = sc.collection.query(query_embeddings=query_vector, n_results=k)

    passages = []
    for doc, meta in zip(results['documents'][0], results['metadatas'][0]):
        sc.retrieved_papers.add(meta['paper'])  # used for the "source retrieved"
metric
        passages.append(f'<passage paper="{meta["paper"]}">\n{doc}\n</passage>')
    return '\n\n'.join(passages) or 'No results found.'
```

# Connecting our agent to our RAG database



***Write a python script to ingest the collected papers into the chroma database (.chroma) using the default embedding model***



***Run the Python script and create the chroma database***

# Configuring our Claude/Pi agent

```
{
  "mcpServers": {
    "chroma": {
      "type": "stdio",
      "command": "uvx",
      "args": [
        "chroma-mcp",
        "--client-type",
        "persistent",
        "--data-dir",
        "/absolute/path/to/project/chroma"
      ]
    }
  }
```

`.pi/mcp.json (Pi)`

`.mcp.json (Claude)`



***mcp list***

THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

# Prompting, RAG, fine-tuning, …?

| Approach | When to use | Watch out for |
|---|---|---|
| **Prompting / context** | First thing to try; information is small and stable | Context limits; cost on every call |
| **RAG** | Knowledge changes often; you need citations; data is private or huge | Answers are capped by retrieval quality |
| **Fine-tuning** | You need consistent style, format, or domain behavior | Needs many good examples; unreliable for teaching new facts; retrain to update |
| **Pretraining** | Almost never your answer | Enormous data and compute |

# Summary: Retrieval Augmented Generation

- RAG brings outside knowledge in: fetch relevant chunks at question time and add them to the prompt
- Embeddings capture meaning: similar text lands nearby, and queries must use the same model and prefixes as the documents
- Use the metric the model was trained for, usually cosine on normalized vectors
- ANN indexes make search scale (recall vs speed)
- Answers are capped by retrieval quality, so tune chunking, k, and reranking

# Part 2 · Memory

# The need for memory

- LLMs are stateless: each session starts with a blank context window
- The context window is the “working memory”

  → limited size and costly

# Scenario 1: long conversations

You've been talking with an agent for days, weeks,... It needs to refer back to earlier sessions



*“I ran into Alok today and he was talking about our project from last month.”*

**To reply, the agent has to recall:**

- Who is Alok, and how is he related to me?
- What was the project, and how did it turn out?
- What is my job?

# Scenario 2: long-running campaigns

A campaign runs for days to weeks and makes thousands of tool calls. New hypotheses and trials must build on earlier results.

### Protocol

The user asked for 3 seeds × 5 configurations to validate results.

### Environment

The simulation needed a specific scheduler flag to finish last time

### Search history

Molecule X was already tried. Don't try it again

### Insight

Adding group Z improved the property before. Try adding it again

# Memory scaling

**Agents should keep improving …**

- Recall earlier sessions: ambiguous references, previous trials
- Repeat less from session to session
- Find the “happy path” faster
- Spend fewer tokens re-reading long context
- Sidestep lost-in-the-middle and other context quirks

# Types of memory

## Episodic

Raw logs: conversations, tool calls, tool results. What happened.

Strategy: store previous session files.

*compact* →

## Semantic

Generalized takeaways from those interactions. What we learned.

Strategy: compact conversations into usable insights.

*Episodic memory is cheap to write / expensive to use. Semantic memory is the opposite.*



THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

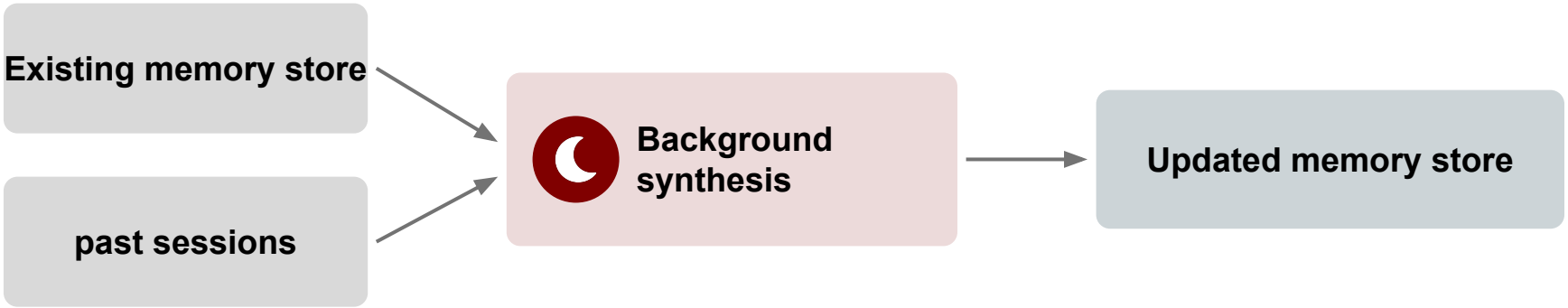
# Coding agent memory in Claude Code

| | CLAUDE.md | Auto memory |
|---|---|---|
| **Written by** | You (or `/init`) | Claude |
| **Contains** | Rules, standards, workflows | Learnings, corrections, preferences |
| **Location** | Repo root and subfolders | `~/.claude/projects/<project>/memory/` |
| **At session start** | Root file loads in full | Only the `MEMORY.md` index loads |
| **Size limit** | None, but shorter is followed better | First 200 lines or 25KB of the index |
| **Best for** | Team-wide rules | Personal, evolving knowledge |
| **Manage with** | Edit the file directly | `/memory` or edit the files |



THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

# Extracting memories via dreaming




Merges duplicates, removes stale notes, keeps the index small

# Picking up where we left off

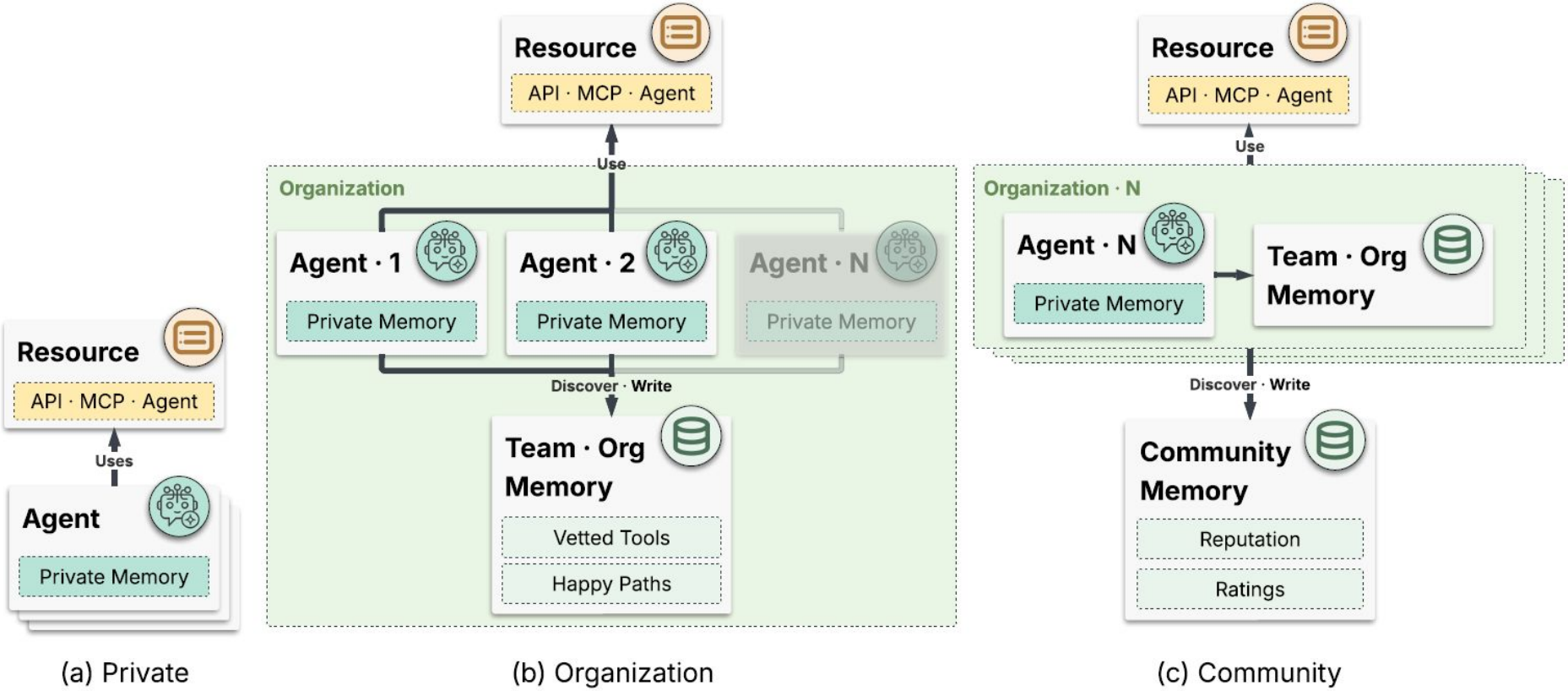
- A markdown file (HANDOFF.md) the agent writes at the end of a session
- Lets a fresh session resume work without re-explaining anything
- Captures the goal, progress, failed approaches, decisions, and next steps
- Failed approaches are the most valuable part, so the next session doesn’t repeat them
- HANDOFF is for short-term task memory; (CLAUDE.md holds the long-term rules)
- Review it before handing off, because the next agent trusts it completely

# Pattern: compaction and handoff files

- Ask the agent to summarize the state so a later session can resume
- Use a template to specify which insights and information are necessary.
- **The template is your answer to “what should we store?”**

```
# HANDOFF.md
## Goal
## Current state
## Decisions and why
## What failed (don't retry)
## Open questions
## Next steps
```

# Sharing memory

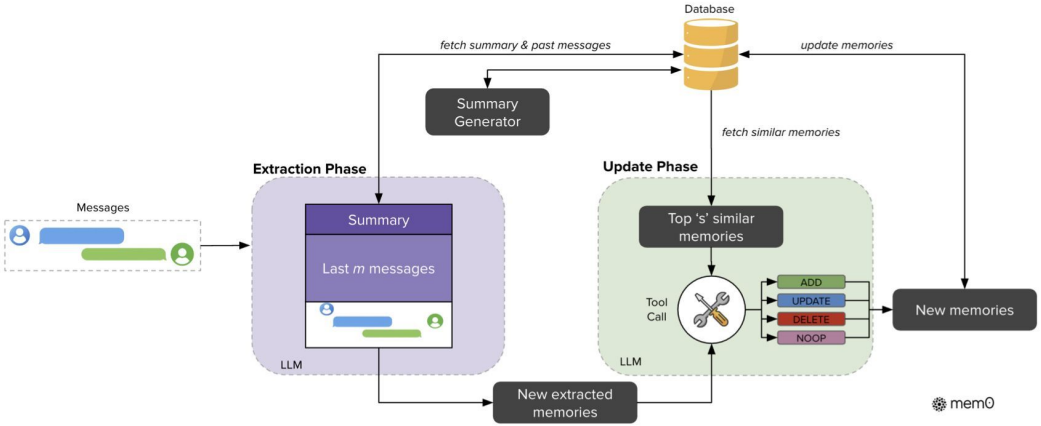


# Example memory system: Mem0

**Extraction:** find salient facts in the new message, using a summary plus the last m messages.

**Update:** fetch the most similar stored memories, then ADD, UPDATE, DELETE, or do nothing.

Both phases can also work on entities and relations (a graph variant).




**UPDATE and DELETE handle staleness:** the user changed jobs; the hypothesis was disproven. Memory has to be revised, not just appended.

*Chhikara et al., "Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory," arXiv, 2025*

# Optional exercise: hand off to a new session

1. **Start a task:** Ask your agent to begin a multi-step task, such as indexing the papers and answering three questions from the benchmark. Stop it partway through.
2. **Write the handoff:** Ask your agent to write HANDOFF.md using the template: goal, current state, decisions and why, what failed, open questions, and next steps.
3. **Review it:** Read HANDOFF.md and fix anything wrong or missing. The next session will trust it completely.
4. **Resume in a fresh session:** Quit, start a new session, and say only: “Read HANDOFF.md and continue.”
5. **Compare:** Did it pick up where it left off? Did it repeat anything that failed? What would you add to the template next time?

# Memory Summary

- LLMs are stateless and every session starts blank
- Types of *memory*:
  a. CLAUDE.md: long-term rules that a human writes and the team shares
  b. Auto memory: learnings the agent writes for itself as it works
  c. Handoff files: short-term state that carries one task between sessions
- Dreaming: background cleanup that keeps memory accurate and compact
- Frameworks like Mem0 add extraction, search, and updates at scale
- Memory needs maintenance. Stale, contradictory, or poisoned notes cause bad behavior

# Mark your calendars: December 8 & 10

**How agents improve**

- Define good science problems for agentic systems
- Measure your agent: test it, break it
- Improve it systematically by tuning
- Make it trustworthy: consider attack vectors and defense mechanisms
- Scale our agentic systems to many agents
- Use agents in your applications!